

MEDIATEK

MT8668 Hypervisor System LLA User Manual

版本: 1.1
出版日期: 2026-03-30

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

版本记录

版本	日期	作者	描述
1.0	2026-01-28	王方方	正式版
1.1	2026-03-30	王方方	移除虚拟化 Vendor 信息

目录

版本记录 2

目录 3

图片目录 4

表格目录 4

1 Hypervisor System 5

 1.1 Build 命令 5

 1.1.1 Hypervisor Android 编译 5

 1.1.2 Hypervisor Yocto 编译 6

 1.1.3 Hypervisor Images 构成 6

 1.2 缩略词 8

 1.3 架构/进程概述 9

 1.3.1 MT8668 Hypervisor 软件架构 9

 1.4 常见问题/故障排除 9

 1.4.1 MT8668 平台 DB 分析指引 9

 1.4.2 内存泄漏问题处理方法 19

 1.4.3 性能模式 22

 1.4.4 CPU 插拔 24

 1.4.5 系统常见问题相关参考 24

附件一 附加条款 26

图片目录

图 1-1. MT8668 Hypervisor 软件架构..... 9

图 1-2. GAT Tool 主界面 10

图 1-3. DBViewer 弹出界面..... 10

图 1-4. DB 解析后生成的文件..... 11

图 1-5. 对解压后生成的文件夹执行 NE/KE 分析..... 13

图 1-6. DB kernel log 13

图 1-7. 内存泄漏示意图..... 19

图 1-8. cat /proc/meminfo 日志..... 20

图 1-9. Kernel memory leak 日志 20

图 1-10. Userspace memory leak 示例..... 21

图 1-11. Lost RAM leak 示例..... 21

表格目录

表 1-1. SOS Yocto 镜像文件..... 6

表 1-2. UOS Android 镜像文件..... 7

表 1-3. 缩略词 8

1 Hypervisor System

MT8668 Hypervisor system 由 Yocto sos、yocto uos、Android 和 Thyp SDK 四大部分组成。

1.1 Build 命令

对于 Hypervisor 3os, 需要编译 Yocto auto8668p1_64_sos、auto8668p1_64_uos 和 Android auto8668p1_64_vm_uos。编译指令如 1.1.1 和 1.1.2 章节所示。由于编译过程中会设置一些环境变量, 为了避免 Yocto 和 Android 互相干扰, 建议使用两个不同的终端分别编译 Yocto 和 Android。Yocto 的编译产物: SOS 和 UOS 在 build/tmp/deploy/auto8668p1_64_sos 目录下。Android 的编译产物在 out/target/product/auto8668p1_64_vm_uos/merged 目录下。编译完成后, 请将 Android 生成的 dtbo.img、boot.img、vendor_boot.img、init_boot.img、userdata.img、super.img、vbmeta.img、vbmeta_system.img、vbmeta_vendor.img、tee.img、scp.img 这十一个 images 复制到 SOS 的编译目录。

1.1.1 Hypervisor Android 编译

MT8668 使用 LD2.0 build, 请参考 MOL: <https://online.mediatek.com/apps/quickstart/QS00266>。

Build 命令:

```
Python vendor/mediatek/proprietary/scripts/releasetools/split_build_helper.py -run  
full_${Project}-${mode}
```

例:

```
Python vendor/mediatek/proprietary/scripts/releasetools/split_build_helper.py -run  
full_auto8668p1_64_vm_uos-userdebug
```

输出: 生成目录 out/target/product/\${project}/merged

LD2.0 会产生 out_sys & out_hal & out_krn & out 这些文件夹用来存放每个叠层的图片。

详情请参考: <https://online.mediatek.com/apps/quickstart/QS00178>。

查询 Build 命令:

```
Python vendor/mediatek/proprietary/scripts/releasetools/split_build_helper.py  
full_${Project}-${mode}
```

1.1.2 Hypervisor Yocto 编译

SOS and UOS build:

```
export TEMPLATECONF=${PWD}/meta/meta-mediatek-mt8668-hyp/conf/templates/auto8668p1_64_sos;
source meta/poky/oe-init-build-env;
bitbake mtk-core-image-auto8668 2>&1 | tee build.log'
```

1.1.3 Hypervisor Images 构成

3os load 烧录一共需要以下 images: bl2.img、vbmeta.img、vbmeta_system.img、vbmeta_vendor.img、modem.img、spmfw.img、mcf_ota.img、pi_img.img、dpm.img、scp.img、vcp.img、sspm.img、mcupm.img、gpueb.img、apusys.img、gz.img、yocto-boot.img、vendor_boot.img、init_boot.img、dtbo.img、tee.img、connsys_bt.img、connsys_wifi.img、connsys_gnss.img、logo.img、audio_dsp.img、yocto_system.img、super.img、userdata.ext4、bl2-an.img、boot.img、ota_prefs.ext4、userdate.img、system_uos.img、userdata_uos.ext4、bl2-uos.img、boot_uos.img。其中 dtbo.img、boot.img、vendor_boot.img、init_boot.img、userdata.img、super.img、vbmeta.img、vbmeta_system.img、vbmeta_vendor.img、tee.img、scp.img 这十一个 images 是使用 Android code build 出来的，其余是由 Yocto code build 产出。

表 1-1. SOS Yocto 镜像文件

Partition_Name	Download_File	说明
preloader_a / preloader_b	bl2.img	Yocto bootloader 分区。由 lk2.bb 编译出来
bl2-an	bl2-an.img	Android bootloader，由 lk2-an.bb 编译出来
pgpt	NONE	GPT 分区表
misc	NONE	存储 OTA 升级过程中的一个标志位
para	NONE	存储 LK 和 kernel 使用的系统环境信息
yocto-expdb	NONE	用于存储 Yocto AEE DB
expdb	NONE	用于存储 Android AEE DB
nvcfg	NONE	由各 module 直接透过 AP side FS 去存放数据
nvdta	NONE	存储 NVRAM 数据
protect1	NONE	存储 SIM lock 数据
protect2	NONE	存储 SIM lock 数据备份
seccfg	NONE	存储安全启动相关的信息

Partition_Name	Download_File	说明
otp	NONE	eMMC OTP (one-time-program, e.g. IMEI)
pstore	NONE	保存一些额外的日志, 比如 power off charge
modem_a	modem.img	modem 镜像
spmfw_a	spmfw.img	spmfw 镜像
mcf_ota_a	mcf_ota.img	mfc_ota 镜像
audio_dsp_a	audio_dsp.img	audio_dsp 镜像
pi_img_a	pi_img.img	pi_img 镜像
dpm_a	dpm.img	dmp 镜像
ccu_a	ccu.img	CCU 镜像
vcp_a	vcp.img	VCP 镜像
sspm_a	sspm.img	SSPM 镜像
mcupm_a	mcupm.img	MCUPM 镜像
gpueb_a	gpueb.img	GPUEB 镜像
apusys_a	apusys.img	APUSYS 镜像
yocto-boot_a	boot.img	SOS Kernel 镜像
connsys_bt_a	connsys_bt.img	GNSS 镜像
connsys_wifi_a	connsys_wifi.img	Wifi fw image
connsys_gnss_a	connsys_gnss.img	Wifi gnss image
logo_a	logo.img	Logo 镜像
yocto_system	yocto_system.img	rootfs 镜像
yocto-userdata	userdata.ext4	SOS 用户数据
ota_prefs	ota_prefs.ext4	存储 OTA 升级过程中的升级状态信息

表 1-2. UOS Android 镜像文件

Partition_Name	Download_File	说明
vbmeta_a	vbmeta.img	system 和 vendor layer 一起的 meta data
vbmeta_system_a	vbmeta_system.img	system layer 的 meta data
vbmeta_vendor_a	vbmeta_vendor.img	vendor system layer 的 meta data
boot_a	boot.img	Android kernel 镜像
vendor_boot_a	vendor_boot.img	Android ramdisk ko 镜像

Partition_Name	Download_File	说明
init_boot_a	init_boot.img	Android init 镜像
dtbo_a	dtbo.img	Android dtb overlay 镜像
tee_a	tee.img	ATF
super	super.img	Android super 镜像
userdata	userdata.img	Android userdata 分区
scp_a	scp.img	Android scp 分区
system_uos_a	system_uos.img	yocto uos system 分区
userdata_uos	userdata_uos.ext4	yocto uos 用户数据
bl2-uos_a	bl2-Yocto UOS.img	yocto uos bl2 分区
boot_uos_a	boot_uos_Yocto UOS.img	yocto uos kernel 镜像

1.2 缩略词

表 1-3. 缩略词

缩略词	全称及释义
AED	Android Exception Daemon 异常守护程序
ANR	Application Not Response 应用程序无响应
ATF	Arm Trusted Firmware Arm 可信固件
EE	External Exception 外部异常
JE	Java Exception Java 异常
KE	Kernel Exception 内核异常
NE	Native Exception Native 异常
Nebula	Vendor 虚拟机微内核
OOM	Out of Memory 内存溢出
PSCI	Power State Coordination Interface 电源状态协调接口
SMC	Secure Monitor Call 安全监控调用
SOS	Service OS 服务操作系统
SWT	Software Watchdog Timeout 软件看门狗超时
Yocto UOS	Yocto User OS 用户操作系统

1.3 架构/进程概述

1.3.1 MT8668 Hypervisor 软件架构

MT8668 Hypervisor 软件架构如图 1-1 所示，主要由三大部分组成。第一个是 vendor 的 Hypervisor 和 Nebula 微内核。第二个是 SOS (Cockpit)，作为主机，运行着虚拟机管理程序，以及虚拟化后端驱动。绝大部分的硬件是直通到 SOS，由 Yocto 构建出来。第三个是 Guest (Android (IVI) /Yocto UOS)，作为客户机。运行着虚拟化前端驱动。只有少量硬件直通到 Guest。

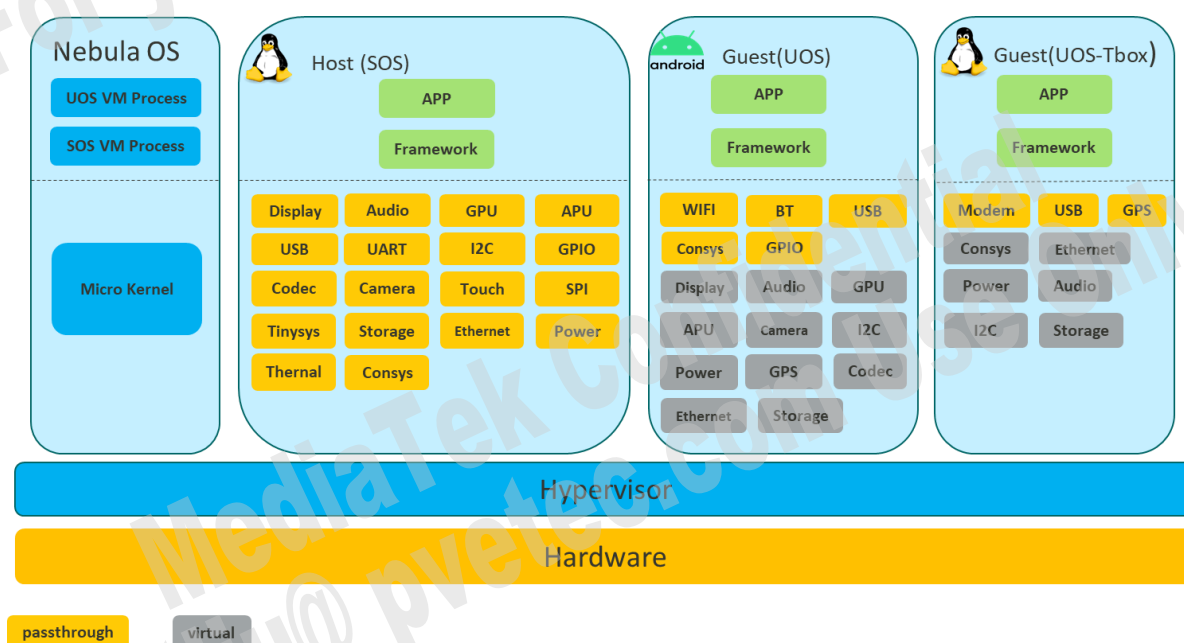


图 1-1. MT8668 Hypervisor 软件架构

1.4 常见问题/故障排除

1.4.1 MT8668 平台 DB 分析指引

1.4.1.1 AEE 介绍

整体与单系统 AEE 类似，本章节主要介绍 Hypervisor 特有的一些信息。

1.4.1.2 DB 保存路径

AEE DB 保存到如下路径中：

SOS:

/data/aee_exp

Android UOS:

/data/aee_exp

/data/vendor/aee_exp

Yocto UOS:

/data/aee_exp

1.4.1.3 Tools

1.4.1.3.1 GAT

1. 打开 GAT 工具。
2. 选择工具栏 Window→Open DBviewer。

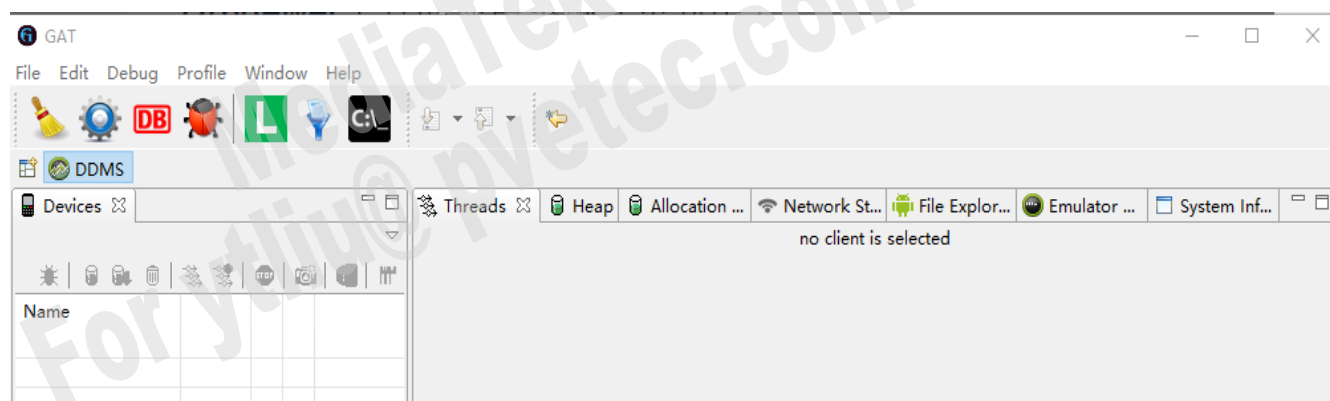


图 1-2. GAT Tool 主界面

3. DBViewer 弹出界面，如下图：

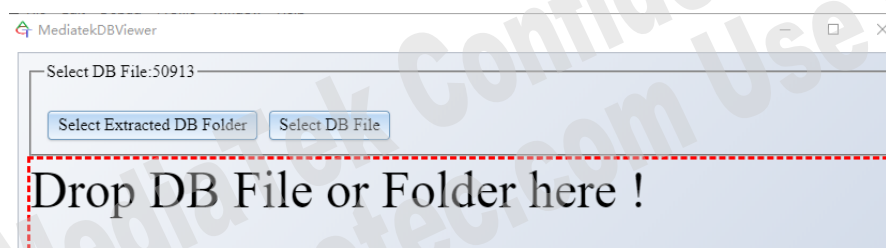


图 1-3. DBViewer 弹出界面

4. 选择 Select DB File，载入要解析的 DB 文件，如：db.fatal.00.KE.dbg。点击 Start，解析完成后，可看到解析出来的 DB Files。可在 dbg 文件同一路径下看到对应文件夹。

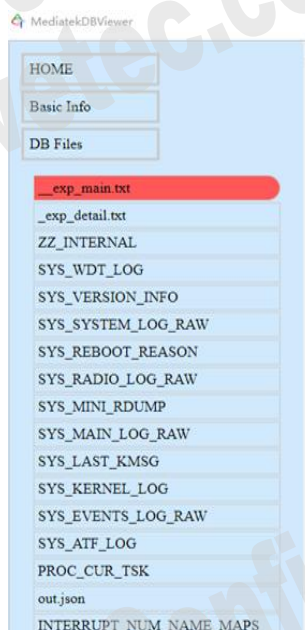


图 1-4. DB 解析后生成的文件

1.4.1.3.2 SpOfflineDebugSuite Tool

SpOfflineDebugSuite tool 用于解析 Android NE/KE DB，也可以用于解析 KE/HWT DB。

注意：SP_OfflineDebugSuite tool 是解析 DB 的工具，并非解压 DB 的工具，需要对 GAT tool 解压后生成的 db.xxx.dbg.DEC 目录文件右键且选择 NE/KE-Analyze。

1.4.1.4 如何分析 KE 问题

- KE：Kernel Exception，大多是 kernel panic/oops 问题。但是对于 LK crash 和 ATF crash 等异常重启问题，也归为 KE DB，具体要查看 DB 内容分析。
- KE DB 文件：db.fatal.00.KE，如果发生 KE，一般会在 /data/vendor/aee_exp 中生成 DB 文件。
- 解压 db.fatal.00.KE.dbg 得到 db.fatal.00.KE.DBG.DEC。
- 解压开的 DB 内容，重点关注 _exp_main.txt。SYS_KERNEL_LOG 信息最多，除了异常信息之外还有异常之前的一些 kernel 日志；SYS_LAST_KMSG 记录重启前最后的日志。
- kernel panic/oops 可以在 kernel log 中看到明显异常 log。

- 查看 KE DB 中的 SYS_CUR_PLBK 文件，搜索 panic_header.exp_type:

panic_header.exp_type=0x2 → Kernel panic

panic_header.exp_type=0x6 → LK crash

panic_header.exp_type=0x9 → TF-A crash

panic_header.exp_type=0xc → Nebula kernel panic

1.4.1.4.1 KE DB 中主要文件

1. __exp_main.txt

记录 1) SW version; 2) Kernel crash core backtrace

Exception Class: Kernel (KE)

PC is at [<ffffffffff310bb01e4>] ipanic+0x78/0xa0 [mrdump]

LR is at [<ffffffffff3118e2168>] notifier_call_chain+0x90/0x174

Backtrace:

```
[<ffffffffff3118e25e0>] atomic_notifier_call_chain+0x38/0x60
[<ffffffffff31286b328>] panic+0x16c/0x35c
[<ffffffffff31208fa08>] sysrq_handle_crash+0x20/0x24
[<ffffffffff31208f5e0>] __handle_sysrq+0x17c/0x188
[<ffffffffff312090844>] write_sysrq_trigger+0xd4/0x128
[<ffffffffff311c52784>] proc_reg_write+0xbc/0x138
[<ffffffffff311ba6ae4>] vfs_write+0x104/0x314
[<ffffffffff311ba6e7c>] ksys_write+0x74/0xe8
[<ffffffffff311ba6f08>] __arm64_sys_write+0x18/0x2c
[<ffffffffff31182df80>] invoke_syscall+0x54/0x114
[<ffffffffff31182dea4>] el0_svc_common+0x7c/0xdc
[<ffffffffff31182de18>] do_el0_svc+0x18/0x28
[<ffffffffff3128723dc>] el0_svc+0x34/0x68
[<ffffffffff312872358>] el0t_64_sync_handler+0x6c/0xbc
[<ffffffffff311811584>] el0t_64_sync+0x1a4/0x1ac
```

2. SYS_KERNEL_LOG_XXX

保存 printk ring buffer 内容，需要解析后才可产生 SYS_KERNEL_LOG，查看重启前的当前 OS kernel log

3. SYS_MINI_RDUMP

类似 coredump 文件，解析后可以产生每个 vCPU 的 callstack

4. SYS_EXP_PL_LK

保存异常重启前后多份 LK log，可以确认 LK 阶段有无 crash 发生

5. TFA_DEBUG_BUF

保存异常重启前的 TF-A log，可以确认 TF-A 有无异常

6. SYS_NEBULA_LOG_DUMP_T (Yocto SOS 异常才会产生这个文件)

Hypervisor 特有的文件，保存异常重启前的 nebula OS log，可以确认 nebula OS 有无异常

7. SYS_UOS_TBOX_PSTORE_RAW (Yocto SOS 异常会产生这个文件)

Hypervisor 特有的文件，Yocto VM 异常时，该文件保存 Yocto UOS VM kernel 和 userspace log

8. SYS_ANDROID_PSTORE_RAW (Yocto SOS 异常会产生这个文件)

Hypervisor 特有的文件，解析出 Yocto VM 异常时的 Android VM kernel log (SYS_ANDROID_VM_LAST_KMSG_k) 和 Android userspace log (SYS_PSTORE_ANDROID_VM_LOG)

9. SYS_YOCTO_PSTORE_RAW (UOS 异常会产生这个文件)

Hypervisor 特有的文件，解析出 Android VM 异常时的 Yocto VM kernel log (SYS_YOCTO_VM_LAST_KMSG_k)

1.4.1.4.2 如何查看重启前的 Kernel Log

SYS_KERNEL_LOG 记录重启前的 kernel log。注意此文件需要右键 NE/KE-Analyze 或者 GAT Analyze 后才会产生。



图 1-5. 对解压后生成的文件夹执行 NE/KE 分析

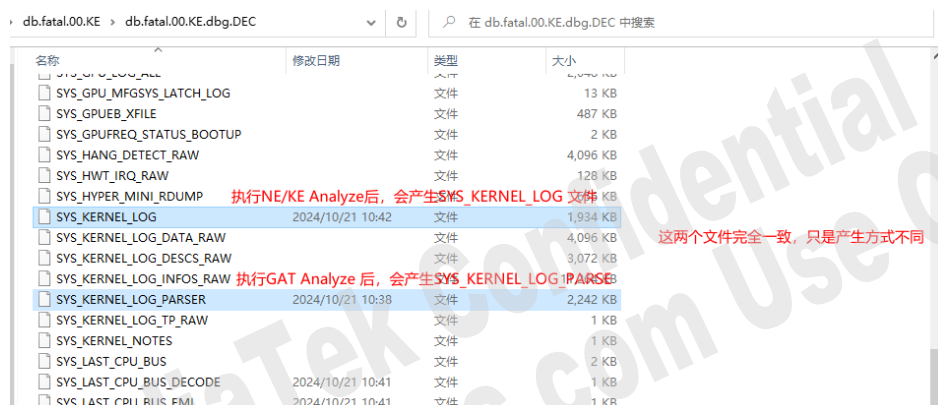


图 1-6. DB kernel log

1.4.1.4.3 如何查看每个 vCPU Trace

使用 NE/KE-Analyze 会自动解析 SYS_MINI_RDUMP 文件生成分析报告，其中会列出每个 vCPU trace。

《_分析报告.txt》中列出的 trace 只有 function name，无具体 line，分析报告最后会列出依赖的 symbol 文件，找到相应的 symbol 文件并放在 db_fatal.xx.KE.dbg.DEC 目录下的 symbols 目录中，重新右键 NE/KE-Analyze 后产生的分析报告中会有具体 line。

1.4.1.5 如何分析 HWT 问题

HWT DB 表示 CPU 卡死触发 watchdog timeout，造成 HWT。

开机阶段 wdk 会初始化好，而且是优先级最高的实时进程，其他任何进程都无法卡到喂狗进程。

谁可以卡喂狗进程？

- 长时间关闭中断。这个直接无法调度，还可以细分为：
 - 关闭中断出现 CPU 死锁
 - 关闭中断后陷入循环
- 中断频繁触发。则导致 CPU 超载，进程基本没有机会被执行到。
- 其他 HW 故障，如总线卡死或 HW 不稳定。

引入多系统后，Yocto SOS 异常导致的 vCPU 卡死，Nebula 异常导致的 pCPU 卡死都会产生 HWT DB。目前 WDT 下沉 Nebula 方案后，可以分析是哪个 OS 触发的 HWT。

查看 HWT DB 中的 SYS_CUR_PLBK 文件

panic_header.wdt_status = 0x2/panic_header.exp_type = 0x1 → Yocto SOS 检测到 vCPU 卡死触发 HWT

panic_header.wdt_status = 0x6/panic_header.exp_type = 0x1 → Nebula OS 检测到 pCPU 卡死触发 HWT

panic_header.wdt_status = 0x2/panic_header.exp_type = 0xd → Nebula OS 检测到 Yocto SOS 所有的 vCPU 卡死触发 HWT

注意：由于 CPU 是共享的，如果检测到 Yocto vCPU 卡死触发 HWT，并不能说明和 Nebula 无关，还需具体情况分析，是否为 Nebula 异常导致 Yocto vCPU 卡死。

1.4.1.5.1 HWT DB 中主要文件

1. SYS_KERNEL_LOG_xxx
保存 printk ring buffer 内容，需要解析后才可产生 SYS_KERNEL_LOG，查看重启前的当前 OS kernel log。确认 wdtk/watchdogd 相关 log
2. SYS_HWT_IRQ_RAW
保存异常重启前最后两次 interrupt 信息
3. SYS_MINI_RDUMP
类似 coredump 文件，解析后可以产生每个 vCPU callstack
4. SYS_HYPER_MINI_RDUMP (Yocto SOS 异常才会产生这个文件)
Hypervisor 特有的文件，gdb 解析后可以产生异常发生时运行在 Nebula OS 的 pCPU callstack
5. SYS_NEBULA_LOG_DUMP_T (Yocto SOS 异常才会产生这个文件)
Hypervisor 特有的文件，保存异常重启前的 Nebula OS log, 可以确认 Nebula OS 有无异常
6. SYS_ANDROID_PSTORE_RAW (Yocto SOS 异常会产生这个文件)
Hypervisor 特有的文件，Yocto VM 异常时，该文件解析出 Android VM kernel log (SYS_ANDROID_VM_LAST_KMSG_k) 和 Android userspace log (SYS_PSTORE_ANDROID_VM_LOG)
7. SYS_UOS_TBOX_PSTORE_RAW (Yocto SOS 异常会产生这个文件)
Hypervisor 特有的文件，Yocto VM 异常时，该文件保存 Yocto UOS VM kernel 和 userspace log
8. SYS_YOCTO_PSTORE_RAW (UOS 异常会产生这个文件)
Hypervisor 特有的文件，解析出 UOS VM 异常时，Yocto SOS kernel log (SYS_YOCTO_VM_LAST_KMSG_k)
9. SYS_LAST_CPU_BUS (Yocto SOS 异常才会产生这个文件)
是否有 tracker/lastbus timeout
10. DFD_SOC.dfd (Yocto SOS 异常才会产生这个文件)
执行 GAT Analyze 后，将 DFD_ANALYSIS \Table.xls->CPC 中 CPC_TRACE 贴到 mcdi_translate_kernel_time.xlsx，得到 kernel time，然后看哪些 CPU 一直没有唤醒
11. TFA_DEBUG_BUF
保存异常重启前的 TF-A log，TF-A log 中会打印重启前 el1 各个 pCPU register 信息

```
NOTICE: (0)K:[155.204897]cpu(0) pc:<ffffffffffd4804877e0> lr:<ffffffffffd4804877dc>
sp:<ffffffffffc0871ebe30> pstate=234000c5
NOTICE: (0)K:[155.205264]cpu(0) sctlr_el1:20000383474d99d tcr_el1:5500072b5593519
ttbr0_el1:19880000e69b9001 ttbr1_el1:198800008182a001
NOTICE: (0)K:[155.205692]cpu(0) sp_el0=ffffffffff8006f7b900 sp_el1=ffffffffffc0871ebe30
NOTICE: (0)K:[155.205954]cpu(0) esr_el2:5a000004 far_el2:ffffffffffc08004014c
hcr_el2:1000300800c263d
NOTICE: (0)K:[155.206270]cpu(0) sctlr_el2:30c5183d tcr_el2:80823510 ttbr0_el2:93830000
elr_el2:ffffffffffd4804877e0
NOTICE: (0)K:[155.206626]cpu(1) pc:<ffffffffffd4804877e0> lr:<ffffffffffd4804877dc>
sp:<ffffffffffc0a9963e30> pstate=234000c5
NOTICE: (0)K:[155.206986]cpu(1) sctlr_el1:20000383474d99d tcr_el1:5500072b5593519
ttbr0_el1:1a9c000108618001 ttbr1_el1:1a9c00008182a001
NOTICE: (0)K:[155.207406]cpu(1) sp_el0=ffffffffff8021f90000 sp_el1=ffffffffffc0a9963e30
NOTICE: (0)K:[155.207664]cpu(1) esr_el2:5a000004 far_el2:ffffffffffc08004014c
hcr_el2:1000300800c263d
NOTICE: (0)K:[155.207979]cpu(1) sctlr_el2:30c5183d tcr_el2:80823510 ttbr0_el2:93830000
elr_el2:ffffffffffd4804877e0
NOTICE: (0)K:[155.208338]cpu(2) pc:<ffffffffff00002bf8> lr:<ffffffffff00024c70>
sp:<ffff000093e61e10> pstate=40000305
NOTICE: (0)K:[155.208704]cpu(2) sctlr_el1:34d5d99d tcr_el1:12b5503590
ttbr0_el1:e000095570000 ttbr1_el1:93dc7000
NOTICE: (0)K:[155.209065]cpu(2) sp_el0=0 sp_el1=ffff000093e61e10
NOTICE: (0)K:[155.209284]cpu(2) esr_el2:5a000004 far_el2:ffffffffffc08004014c
hcr_el2:1000300800c263d
NOTICE: (0)K:[155.209599]cpu(2) sctlr_el2:30c5183d tcr_el2:80823510 ttbr0_el2:93830000
elr_el2:ffffffffffd4804877e0
NOTICE: (0)K:[155.209958]cpu(3) pc:<ffffffffff00002bf8> lr:<ffffffffff00024c70>
sp:<ffff000093e65e30> pstate=40000305
NOTICE: (0)K:[155.210322]cpu(3) sctlr_el1:34d5d99d tcr_el1:12b5503590
ttbr0_el1:e000095570000 ttbr1_el1:93dc7000
NOTICE: (0)K:[155.210683]cpu(3) sp_el0=0 sp_el1=ffff000093e65e30
NOTICE: (0)K:[155.210902]cpu(3) esr_el2:5a000004 far_el2:ffffffffffc08004014c
hcr_el2:1000300800c263d
NOTICE: (0)K:[155.211217]cpu(3) sctlr_el2:30c5183d tcr_el2:80823510 ttbr0_el2:93830000
elr_el2:ffffffffffd4804877e0
NOTICE: (0)K:[155.211571]cpu(4) pc:<ffffffffffd4804877e0> lr:<ffffffffffd4804877dc>
sp:<ffffffffffc0871d3e30> pstate=234000c5
NOTICE: (0)K:[155.211931]cpu(4) sctlr_el1:20000383474d99d tcr_el1:5500072b5593519
ttbr0_el1:1c36000108898001 ttbr1_el1:1c3600008182a001
NOTICE: (0)K:[155.212359]cpu(4) sp_el0=ffffffffff80cc891300 sp_el1=ffffffffffc0871d3e30
NOTICE: (0)K:[155.212622]cpu(4) esr_el2:5a000004 far_el2:ffffffffffc08004014c
hcr_el2:1000300800c263d
NOTICE: (0)K:[155.212939]cpu(4) sctlr_el2:30c5183d tcr_el2:80823510 ttbr0_el2:93830000
elr_el2:ffffffffffd4804877e0
NOTICE: (0)K:[155.213298]cpu(5) pc:<ffffffffff00002bf8> lr:<ffffffffff00024c70>
sp:<ffff000093e6de70> pstate=40000305
NOTICE: (0)K:[155.213665]cpu(5) sctlr_el1:34d5d99d tcr_el1:12b5503590
ttbr0_el1:a000095374000 ttbr1_el1:93dc7000
NOTICE: (0)K:[155.214026]cpu(5) sp_el0=0 sp_el1=ffff000093e6de70
NOTICE: (0)K:[155.214243]cpu(5) esr_el2:5a000004 far_el2:ffffffffffc08004014c
hcr_el2:1000300800c263d
NOTICE: (0)K:[155.214560]cpu(5) sctlr_el2:30c5183d tcr_el2:80823510 ttbr0_el2:93830000
elr_el2:ffffffffffd4804877e0
```

```

NOTICE: (0)K:[155.214919]cpu(6) pc:<ffffffff00002bf8> lr:<ffffffff00024c70>
sp:<ffff000093e71e90> pstate=40000305
NOTICE: (0)K:[155.215284]cpu(6) sctlr_el1:34d5d99d tcr_el1:12b5503590
ttbr0_el1:e000095570000 ttbr1_el1:93dc7000
NOTICE: (0)K:[155.215639]cpu(6) sp_el0=0 sp_el1=ffff000093e71e90
NOTICE: (0)K:[155.215854]cpu(6) esr_el2:5a000004 far_el2:ffffffc08004014c
hcr_el2:1000300800c263d
NOTICE: (0)K:[155.216167]cpu(6) sctlr_el2:30c5183d tcr_el2:80823510 ttbr0_el2:93830000
elr_el2:ffffffd4804877e0
NOTICE: (0)K:[155.216526]cpu(7) pc:<ffffffff00002bf8> lr:<ffffffff00024c70>
sp:<ffff000093e75eb0> pstate=40000305
NOTICE: (0)K:[155.216891]cpu(7) sctlr_el1:34d5d99d tcr_el1:12b5503590
ttbr0_el1:a000095374000 ttbr1_el1:93dc7000
NOTICE: (0)K:[155.217252]cpu(7) sp_el0=0 sp_el1=ffff000093e75eb0
NOTICE: (0)K:[155.217471]cpu(7) esr_el2:5a000004 far_el2:ffffffc08004014c
hcr_el2:1000300800c263d
NOTICE: (0)K:[155.217786]cpu(7) sctlr_el2:30c5183d tcr_el2:80823510 ttbr0_el2:93830000
elr_el2:ffffffd4804877e0

```

1.4.1.5.2 查看哪个 vCPU 卡死

从 SYS_KERNEL_LOG 可以看到哪几个 core 没有 kick watchdog，搜索关键字 “kick=”

例如：

```
[T1700707] wdtk-7: kick=0xd4,check=0xff
```

kick=0xd4: 8 bits 对应 core7 ~ core0，表示 CPU0,1,3,5 没有喂狗

1.4.1.5.3 查看 vCPU Callstack

如上述介绍 KE DB 解析 vCPU callstack：用 NE/KE-Analyze 解析 KE DB，根据生成分析报告查看 vCPU callstack。

1.4.1.5.4 查看 HyperOS pCPU Backtrace

检查 SYS_HYPER_MINI_RDUMP。

1.4.1.5.5 HWT Dispatch Rule

对于未踢狗且处于非 idle 状态的 CPU：

- PC 卡在哪个 module，需要相关 module owner 进行检查。

对于未踢狗且处于 WFI 的 CPU:

1. 确认 arch timer 是否有发 IRQ。
2. GIC 是否有收到 IRQ。
3. CPU 是否有 pending IRQ。

对于未踢狗且处于 MCDI 的 CPU:

1. 确认 systimer 是否有发 IRQ。
2. GIC 是否有收到 IRQ。
3. CPC 是否有收到 wakeup 信号。
4. CPC 是否有把 CPU 正常上电。
5. CPU 是否有 pending IRQ。

注意: HWT 问题分析比较麻烦, 以上只是笼统介绍大概分析流程, 对于疑难的 HWT 问题, 还需有经验的 SI/timer/GIC/CPU/power 等 owner 协同分析。

1.4.1.6 如何分析 HWR 问题

HW_Reboot case 不会走异常处理流程, 导致 DB 中通过 SW 方式收集的 core dump 文件无法解析出有效 trace, 需要 HW module 收集的信息定位问题。

1.4.1.6.1 HWR DB 中主要文件

1. SYS_KERNEL_LOG_XXX

保存 printk ring buffer 内容, 需要解析后才可产生 SYS_KERNEL_LOG, 查看重启前的 Yocto kernel log, 确认 wdt/watchdogd 相关 log

2. SYS_LAST_CPU_BUS

是否有 tracker/lastbus timeout 及 last PC, 如:

read entry = 31, valid = 0x1, non-secure = 0x0, read master id = 0x46, slave address = 0x1a078a0c, data_size = 0x2, burst_length = 0xf

表明 slave address = 0x1a078a0c 地址发生 bus hang, 可以通过查看 dts 确认此 address 是属于哪个 module register address 范围

3. DFD_SOC.dfd

GAT Analyze 后得到 DFD_ANALYSIS, 查看 dfd_result.log

4. SYS_CUR_PLLK

异常重启后的 LK log, 搜索 g_rgu_status, 根据 WDT_STA register bits map, 确认重启原因 (WDT_STA bits map 每个 IC 有区别, 可以找 WDT owner 确认)

例如:

以 MT8668 举例, 查看异常重启后的 LK log: parse g_rgu_status: 32788 (0x8014), 发现 bit 2 和 bit4 被置起, 结合 WDT_STA bit maps, 可以确认为 mcu_thermal rst。

1.4.2 内存泄漏问题处理方法

1.4.2.1 内存泄漏简单介绍

内存泄漏 (Memory Leak) 指由于疏忽或错误造成程序未能释放已经不再使用的内存。内存泄漏并非指内存存在物理上的消失, 而是应用程序分配某段内存后, 由于设计错误, 导致在释放该段内存之前就失去了对该段内存的控制, 从而造成了内存的浪费。

判断是否有内存泄漏的关键信息是 memory 占用是否一直在涨。

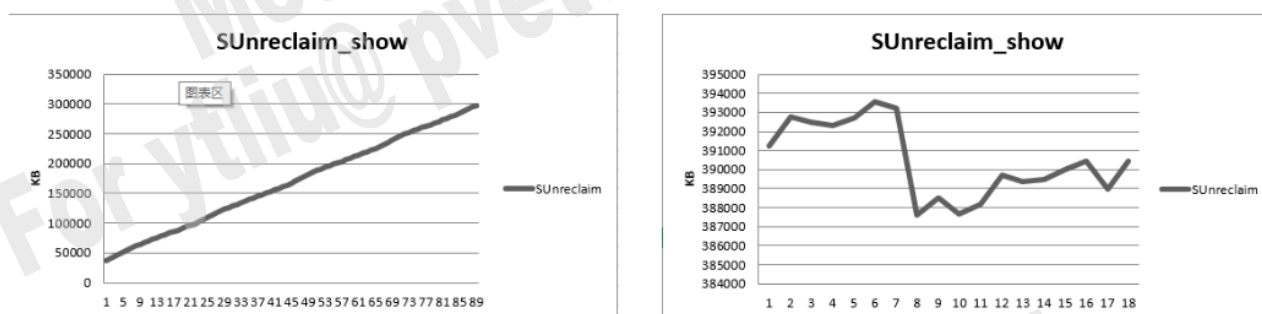


图 1-7. 内存泄漏示意图

- 左图中, slab 的占用一直在上涨, 我们就认为是 leak。
- 右图中, slab 的占用在一个范围内波动, 那么我们就认为这个是正常的波动, 不是 leak。

1.4.2.2 查看 Memory 的命令

- dumphys meminfo 查看 Android 系统详细的 memory 分布。
- meminfo 查看 Yocto 系统详细的 memory 分布。
- cat /proc/meminfo 查看 kernel 统计的 memory 分布。

无论是 Android 还是 Yocto 系统，对于 kernel leak 问题处理方法是一样的。

都是通过命令或者 bat 脚本，在复制问题的同时不断的在后台 dump memory 信息，然后根据 memory 信息定位是哪部分存在 leak，然后借助 tool 进一步分析 leak 是怎么产生的。

1.4.2.3 判断 Kernel Memory Leak 的方式

输入命令：cat /proc/meminfo:

```
MemTotal:      965616 kB
MemFree:       114640 kB
MemAvailable:  491072 kB
Buffers:       7224 kB
Cached:       393032 kB
Shmem:        1184 kB
Slab:         55916 kB
SReclaimable: 18788 kB
SUnreclaim:   37128 kB
KernelStack:  12592 kB
PageTables:   17876 kB
VmallocUsed:   0 kB
```

图 1-8. cat /proc/meminfo 日志

- 如果 sunreclaim 这个值很大且持续增长，基本可以确定 kernel memory leak 发生了，如图 1-8 所示。
- 在 kernel 日志中，如果发生了 OOM (Out of Memory)，也会 dump 出来这部分的 memory。如果看到 sunreclaim 很大且持续增长，基本上也可以判断是有 kernel memory leak，如图 1-9 所示：

```
[147842.604198] slab_reclaimable:12762 slab_unreclaimable:119398
.....
[151361.050385] slab_reclaimable:8474 slab_unreclaimable:122915
.....
[153699.412832] slab_reclaimable:7833 slab_unreclaimable:124784
```

图 1-9. Kernel memory leak 日志

Kernel leak 可以通过开启 kmemleak 或 slub debug 来检查，kmemleak 和 slub debug 是 kernel 自带的 leak tool，可以参考如下资料：

<https://online.mediatek.com/apps/faq/detail?list=SW&faqid=FAQ20884>

<https://online.mediatek.com/apps/quickstart/QS00086#QSS00890>

1.4.2.4 Userspace Memory Leak

Android 使用 `dumpsys meminfo`, Yocto 使用 `meminfo`, 可以看到 userspace 进程的内存使用数据。如果看到进程的 `pss` 一直增长, 则可以认为该进程存在内存泄漏, 如图 1-10 所示。

	PID	Vss	Rss	Pss	Uss	cmdline
Line 636:	1369	2892560K	215072K	212467K	211816K	hal3test
Line 715:	1369	3804008K	260248K	257643K	256992K	hal3test
Line 794:	1369	3804008K	262832K	260227K	259576K	hal3test

图1-10. Userspace memory leak 示例

一般来说, 找到 leak 的进程名称, 就可以找到对应 owner 去检查。

1.4.2.5 Lost RAM Leak

Lost RAM 表示的是系统无法统计的 memory 的数值, 一般是通过直接分配 page 申请的, 其基本的计算方式是:

$$\text{Lost RAM} = \text{Total RAM} - \text{Free RAM} - \text{Used RAM}$$

如果通过 `dumpsys meminfo` 或 `meminfo` 看到 Lost RAM 的数值较大, 且一直不断增长, 则说明 Lost RAM 有 leak, 如图 1-11 所示。

```
Line 248152: Lost_RAM = 5185879
Line 508985: Lost_RAM = 5251825
Line 791384: Lost_RAM = 5297017
Line 1080411: Lost_RAM = 5317652
```

图1-11. Lost RAM leak 示例

对于 Lost RAM leak, 可以开启 page owner 来检查 page 的分配记录、backtrace 等。

Page owner 的使用方法可以参考:

<https://online.mediatek.com/apps/faq/detail?list=SW&faqid=FAQ21615>

1.4.2.6 相关资料参考

内存泄漏专题分析: <https://online.mediatek.com/apps/quickstart/QS00086>

Memory: <http://online.mediatek.com/QuickStart/QS00227>

1.4.3 性能模式

1.4.3.1 性能模式简单介绍

Performance Mode（性能模式）是一种在移动设备、笔记本电脑和台式机等设备上提高系统性能的配置选项。在性能模式下，设备会优先使用更高的 CPU 频率、更大的内存缓存、更高的屏幕刷新率和更高的功率消耗等，以提供更出色的系统性能和响应速度。这种模式通常是在需要高性能运行的时候使用。

1.4.3.2 adb 设置性能模式

1.4.3.2.1 CPU

在 3os 下，Yocto 和 Android 端应该各自输入以下命令：

Yocto：

```
adb -s YOCTO shell "cat /sys/devices/system/cpu/cpufreq/policy7/cpuinfo_max_freq >
/sys/devices/system/cpu/cpufreq/policy7/scaling_max_freq"
adb -s YOCTO shell "cat /sys/devices/system/cpu/cpufreq/policy7/cpuinfo_max_freq >
/sys/devices/system/cpu/cpufreq/policy7/scaling_min_freq"
adb -s YOCTO shell "cat /sys/devices/system/cpu/cpufreq/policy4/cpuinfo_max_freq >
/sys/devices/system/cpu/cpufreq/policy4/scaling_max_freq"
adb -s YOCTO shell "cat /sys/devices/system/cpu/cpufreq/policy4/cpuinfo_max_freq >
/sys/devices/system/cpu/cpufreq/policy4/scaling_min_freq"
adb -s YOCTO shell "cat /sys/devices/system/cpu/cpufreq/policy0/cpuinfo_max_freq >
/sys/devices/system/cpu/cpufreq/policy0/scaling_max_freq"
adb -s YOCTO shell "cat /sys/devices/system/cpu/cpufreq/policy0/cpuinfo_max_freq >
/sys/devices/system/cpu/cpufreq/policy0/scaling_min_freq"
adb -s YOCTO shell "modprobe cpu_hwtest"
adb -s YOCTO shell "echo 2 > /proc/cpuhvfs/cpufreq_cci_mode"
adb -s YOCTO shell "echo 0 > /proc/cpuhvfs/cci_dvfs_enable"
adb -s YOCTO shell "echo 0 > /proc/cpuhvfs/cci_opp_send"
adb -s YOCTO shell "cat /proc/cpuhvfs/phyclk"
```

Android：

```
adb -s 0123456789ABCDEF shell "cat /sys/devices/system/cpu/cpufreq/policy7/cpuinfo_max_freq >
/sys/devices/system/cpu/cpufreq/policy7/scaling_max_freq"
adb -s 0123456789ABCDEF shell "cat /sys/devices/system/cpu/cpufreq/policy7/cpuinfo_max_freq >
/sys/devices/system/cpu/cpufreq/policy7/scaling_min_freq"
adb -s 0123456789ABCDEF shell "cat /sys/devices/system/cpu/cpufreq/policy4/cpuinfo_max_freq >
/sys/devices/system/cpu/cpufreq/policy4/scaling_max_freq"
adb -s 0123456789ABCDEF shell "cat /sys/devices/system/cpu/cpufreq/policy4/cpuinfo_max_freq >
/sys/devices/system/cpu/cpufreq/policy4/scaling_min_freq"
adb -s 0123456789ABCDEF shell "cat /sys/devices/system/cpu/cpufreq/policy0/cpuinfo_max_freq >
/sys/devices/system/cpu/cpufreq/policy0/scaling_max_freq"
adb -s 0123456789ABCDEF shell "cat /sys/devices/system/cpu/cpufreq/policy0/cpuinfo_max_freq >
/sys/devices/system/cpu/cpufreq/policy0/scaling_min_freq"
```

Yocto UOS:

```
adb -s 0123456789ABCDEF shell "cat /sys/devices/system/cpu/cpufreq/policy7/cpuinfo_max_freq > /sys/devices/system/cpu/cpufreq/policy7/scaling_max_freq"
adb -s 0123456789ABCDEF shell "cat /sys/devices/system/cpu/cpufreq/policy7/cpuinfo_max_freq > /sys/devices/system/cpu/cpufreq/policy7/scaling_min_freq"
adb -s 0123456789ABCDEF shell "cat /sys/devices/system/cpu/cpufreq/policy4/cpuinfo_max_freq > /sys/devices/system/cpu/cpufreq/policy4/scaling_max_freq"
adb -s 0123456789ABCDEF shell "cat /sys/devices/system/cpu/cpufreq/policy4/cpuinfo_max_freq > /sys/devices/system/cpu/cpufreq/policy4/scaling_min_freq"
adb -s 0123456789ABCDEF shell "cat /sys/devices/system/cpu/cpufreq/policy0/cpuinfo_max_freq > /sys/devices/system/cpu/cpufreq/policy0/scaling_max_freq"
adb -s 0123456789ABCDEF shell "cat /sys/devices/system/cpu/cpufreq/policy0/cpuinfo_max_freq > /sys/devices/system/cpu/cpufreq/policy0/scaling_min_freq"
```

CPU Query (Both Yocto and Android):

The actual data obtained from the fmeter shall take precedence.

```
adb shell "cat /sys/devices/system/cpu/cpufreq/policy*/scaling_cur_freq"
adb shell "echo fmeter > /proc/clkdbg;cat /proc/clkdbg | grep armpll"
```

1.4.3.2.2 GPU

GPU set:

```
adb shell "echo 0 > /proc/gpufreqv2/fix_target_opp_index"
```

GPU query:

```
adb shell "cat /proc/gpufreqv2/gpufreq_status"
```

1.4.3.2.3 DDR

Set DDR frequency:

```
adb shell "echo 0 > /sys/kernel/helio-dvfsrc/dvfsrc_force_vcore_dvfs_opp"
```

Check DDR frequency:

```
adb shell "cat /sys/kernel/helio-dvfsrc/dvfsrc_dump | grep bps"
```

1.4.3.2.4 Thermal Disable

Yocto:

```
adb -s YOCTO shell "thermal-int apply disable_throttling_and_fan.conf"
```

Android:

```
adb -s 0123456789ABCDEF shell "/vendor/bin/thermal_intf apply disable_throttling.conf"
```

1.4.3.3 在代码中修改开机自启动性能模式

目前 Hypervisor Yocto 和 Hypervisor Android 开机之后默认已经是 CPU performance mode。

Hypervisor Yocto code:

meta/meta-mediatek-mt8668/recipes-bsp/bootop/files/bootop.sh:

```
echo "set cpufreq scaling_governor to performance" > /dev/kmsg
echo performance > /sys/devices/system/cpu/cpufreq/policy0/scaling_governor
echo performance > /sys/devices/system/cpu/cpufreq/policy4/scaling_governor
echo performance > /sys/devices/system/cpu/cpufreq/policy7/scaling_governor
```

Hypervisor Android code:

alps/device/mediateksample/auto8668p1_64_bsp_vm 目录下 init.cgroup-kernel-mainline.rc 和

init.cgroup-kernel-6.1.rc:

```
write /sys/devices/system/cpu/cpufreq/policy0/scaling_governor performance
write /sys/devices/system/cpu/cpufreq/policy4/scaling_governor performance
write /sys/devices/system/cpu/cpufreq/policy7/scaling_governor performance
write /sys/kernel/helio-dvfsrc/dvfsrc_force_vcore_dvfs_opp 255
```

1.4.4 CPU 插拔

您可以用如下指令来开关核:

```
adb shell "echo 1 > /sys/devices/system/cpu/cpu1/online"
adb shell "echo 1 > /sys/devices/system/cpu/cpu2/online"
adb shell "echo 1 > /sys/devices/system/cpu/cpu3/online"
```

echo 1 是打开, echo 0 是关闭。

1.4.5 系统常见问题相关参考

内存泄漏专题分析: <https://online.mediatek.com/apps/quickstart/QS00086>

踩内存专题分析: <https://online.mediatek.com/apps/quickstart/QS00090>

死机问题快速分析: <http://online.mediatek.com/QuickStart/QS00044>

Memory: <http://online.mediatek.com/QuickStart/QS00227>

Performance 调试宝典: <https://online.mediatek.com/apps/quickstart/QS00288>

深入分析 Android native exception 框架: <http://online.mediatek.com/QuickStart/QS00038>

深入分析看门狗框架: <http://online.mediatek.com/QuickStart/QS00067>

Hang Detect 问题快速分析 <https://online.mediatek.com/apps/quickstart/QS00076>

深入分析 Linux kernel exception 框架 <http://online.mediatek.com/QuickStart/QS00073>

NE/KE 分析学习课程 <https://online.mediatek.com/apps/quickstart/QS00068>

NE/KE 分析报告解读 <https://online.mediatek.com/apps/quickstart/QS00089>

附件一 附加条款

您(或您所代表的公司或其他法律主体, 以下合称「您」)是否得取得、下载与使用本文件及其相关信息皆应以您接受本附加条款为先决要件。您对于本文件之使用、取得或下载即表示您已接受本附加条款并同意受本附加条款之拘束。若您不同意受本附加条款之拘束, 您将不得使用、取得或下载本文件并应立即删除或毁弃所有本文件之副本。

本文件含有联发科技股份有限公司及其关联公司(以下合称「联发科技」)或其授权人之机密信息及专有信息, 仅供您为本文件所描述之联发科技芯片组作内部使用而不得用于其他任何目的(包括但不限于确认或提供支持任何对联发科技、其供货商及/或其直接或间接客户所提潜在专利侵权主张的证据)。禁止未经授权使用或揭露本文件及其中所含信息。您同意赔偿联发科技因您未经授权使用或揭露本文件及其中所含信息之一部或全部导致联发科技所受之任何损失或损害。

联发科技及其授权人保有本文件的所有权及组成所有权之各项权利且未针对任何知识产权授权(无论明示或默示、透过禁反言原则或其他方式)。联发科技得随时变更本文件内容而无须另行通知。联发科技无须承担与使用或信赖本文件相关或因使用或信赖本文件致生之任何责任, 包括但不限于间接损害或附带损害赔偿責任。

本文件及联发科技所提供关于本文件之任何其他材料、信息或技术支持, 均以现况交付为准, 联发科技不负任何明示、默示、法定或其他形式之担保责任, 特别是适销性、不侵权、针对特定用途之适用性、完整性或正确性之担保责任或任何由贸易惯例或交易、履行过程所生之担保责任。对于联发科技为符合您所提规格或遵循特定标准或标准组织的要求所作之交付物, 联发科技亦不应承担任何责任。

于未对前述条款造成限制之情形下, 联发科技不对其产品任何特定用途之适用性承担任何保证或担保责任, 亦不承担任何因产品、电路或软件之应用或使用致生之赔偿责任。您同意您应单独承担关于整合您的产品与联发科技产品所涉及之设计、验证与测试之所有责任, 并确保前述整合产品符合相关标准及任何安全性要求或其他要件。

本附加条款及所有与本附加条款或本文件相关之行为应以台湾法律管辖、解释与阐明, 不适用冲突法原则。